



**TECNOLÓGICO NACIONAL DE MÉXICO
INSTITUTO TECNOLÓGICO DE TIJUANA**

**SUBDIRECCIÓN ACADÉMICA
DEPARTAMENTO DE SISTEMAS Y COMPUTACIÓN**

SEMESTRE:
Agosto-Diciembre 2025

CARRERA:
INGENIERÍA EN SISTEMAS COMPUTACIONALES

MATERIA:
Patrones de diseño

TÍTULO ACTIVIDAD :
Examen unidad 2 Patrones de diseño

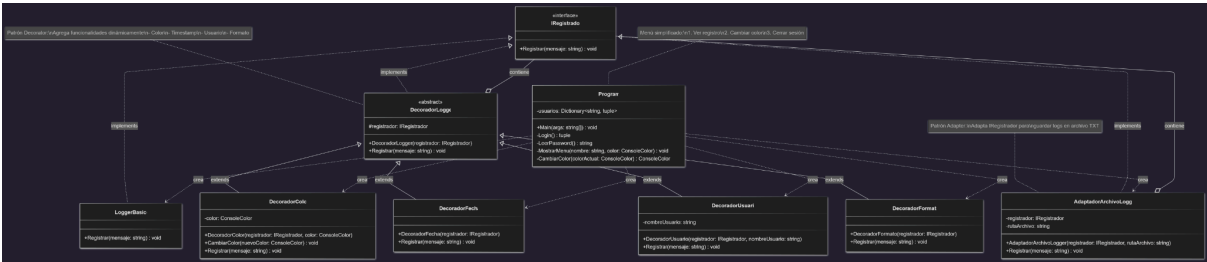
UNIDAD 2

NOMBRE Y NÚMERO DE CONTROL:
NARVAEZ MATA ALEJANDRO- 22210325

NOMBRE DEL MAESTRO (A):

MARIBEL GUERRERO LUIS

UML



Codigo

[adaptadorarchivologger.cs](#)

```
using System;

using System.IO;

namespace ExamenUnidad3
{
    public class AdaptadorArchivoLogger : IRegistrador
    {
        private readonly IRegistrador _registrador;

        private readonly string _rutaArchivo;

        public AdaptadorArchivoLogger(IRegistrador registrador, string rutaArchivo)
        {
            _registrador = registrador;

            _rutaArchivo = rutaArchivo;
        }

        public void Registrar(string mensaje)
        {
            _registrador.Registrar(mensaje);

            string entrada = $"[{DateTime.Now:yyyy-MM-dd HH:mm:ss}] {mensaje}";
```

```
        File.AppendAllText(_rutaArchivo, entrada +  
Environment.NewLine);  
  
    }  
  
}  
  
}
```

[decoradorcolor.cs](#)

```
using System;

namespace ExamenUnidad3
{
    public class DecoradorColor : IRegistrador
    {
        private readonly IRegistrador _registrador;

        private ConsoleColor _color;

        public DecoradorColor(IRegistrador registrador, ConsoleColor
color)
        {
            _registrador = registrador;

            _color = color;
        }

        public void CambiarColor(ConsoleColor nuevoColor)
        {
            _color = nuevoColor;
        }

        public void Registrar(string mensaje)
        {
            Console.ForegroundColor = _color;
```

```
        _registrador.Registrar(mensaje);  
  
        Console.ResetColor();  
    }  
}  
}
```

[decoradorfecha.cs](#)

```
using System;

namespace ExamenUnidad3
{
    public class DecoradorFecha : IRegistrador
    {
        private readonly IRegistrador _registrador;

        public DecoradorFecha(IRegistrador registrador)
        {
            _registrador = registrador;
        }

        public void Registrar(string mensaje)
        {
            string fecha = DateTime.Now.ToString("HH:mm:ss");
            _registrador.Registrar($"[{fecha}] {mensaje}");
        }
    }
}
```

[decoradorformato.cs](#)

```
namespace ExamenUnidad3
{
    public class DecoradorFormato : IRegistrador
    {
        private readonly IRegistrador _registrador;

        public DecoradorFormato(IRegistrador registrador)
        {
            _registrador = registrador;
        }

        public void Registrar(string mensaje)
        {
            _registrador.Registrar($">>> {mensaje}");
        }
    }
}
```


[decoradorusuario.cs](#)

```
namespace ExamenUnidad3
{
    public class DecoradorUsuario : IRegistrador
    {
        private readonly IRegistrador _registrador;

        private readonly string _nombreUsuario;

        public DecoradorUsuario(IRegistrador registrador, string
nombreUsuario)
        {
            _registrador = registrador;

            _nombreUsuario = nombreUsuario;
        }

        public void Registrar(string mensaje)
        {
            _registrador.Registrar($"[{_nombreUsuario}] {mensaje}");
        }
    }
}
```

[iregistrador.cs](#)

```
namespace ExamenUnidad3
{
    public interface IRegistrador
    {
        void Registrar(string mensaje);
    }
}
```

[loggerbasico.cs](#)

```
using System;

namespace ExamenUnidad3
{
    public class LoggerBasico : IRegistrador
    {
        public void Registrar(string mensaje)
        {
            Console.WriteLine(mensaje);
        }
    }
}
```

[program.cs](#)

```
using System;

using System.Collections.Generic;

namespace ExamenUnidad3
{
    class Program
    {
        static Dictionary<string, (string password, ConsoleColor
color)> usuarios = new Dictionary<string, (string, ConsoleColor)>
        {
            { "Aleeeex", ("alexcool1", ConsoleColor.Cyan) },
            { "Juan", ("juan123", ConsoleColor.Green) },
            { "Maria", ("maria456", ConsoleColor.Yellow) }
        };

        static void Main(string[] args)
        {
            bool salirPrograma = false;

            while (!salirPrograma)
            {
                Console.Clear();

                Console.WriteLine("=== Sistema de Logging ===\n");
```

```
        var usuario = Login();

        if (usuario.nombre != null)

        {

            MostrarMenu(usuario.nombre, usuario.color);

        }

        else

        {

            salirPrograma = true;

        }

    }

    Console.WriteLine("\nGracias por usar el sistema.");

    Console.ReadKey();

}

static (string nombre, ConsoleColor color) Login()

{

    Console.Write("Usuario: ");

    string username = Console.ReadLine();

    Console.Write("Contraseña: ");

    string password = LeerPassword();

    if (usuarios.ContainsKey(username) &&
usuarios[username].password == password)
```

```
{

    Console.WriteLine("\n✓ Login exitoso\n");

    return (username, usuarios[username].color);

}

Console.WriteLine("\nX Usuario o contraseña incorrectos");

return (null, ConsoleColor.White);

}

static string LeerPassword()

{

    string password = "";

    ConsoleKeyInfo key;

    do

    {

        key = Console.ReadKey(true);

        if (key.Key != ConsoleKey.Backspace && key.Key !=
ConsoleKey.Enter)

        {

            password += key.KeyChar;

            Console.Write("*");

        }

    }
```

```

        else if (key.Key == ConsoleKey.Backspace &&
password.Length > 0)

        {

            password = password.Substring(0, password.Length -
1);

            Console.Write("\b \b");

        }

    } while (key.Key != ConsoleKey.Enter);

    Console.WriteLine();

    return password;

}

static void MostrarMenu(string nombre, ConsoleColor color)

{

    DecoradorColor decoradorColor = new DecoradorColor(new
LoggerBasico(), color);

    IRegistrador registrador = new AdaptadorArchivoLogger(

        new DecoradorFormato(

            new DecoradorUsuario(

                new DecoradorFecha(decoradorColor), nombre

            )

        ), "logs.txt"

    );

```

```
bool salir = false;

while (!salir)

{

    Console.ForegroundColor = color;

    Console.WriteLine("\n=== MENÚ ===");

    Console.WriteLine("1. Ver registro");

    Console.WriteLine("2. Cambiar color");

    Console.WriteLine("3. Cerrar sesión");

    Console.ResetColor();

    Console.Write("\nOpción: ");

    string opcion = Console.ReadLine();

    switch (opcion)

    {

        case "1":

            registrador.Registrar("Visualizando registro del sistema");

            break;

        case "2":

            color = CambiarColor(color);

            decoradorColor.CambiarColor(color);

            usuarios[nombre] = (usuarios[nombre].password, color);

            registrador.Registrar($"Color cambiado a {color}");
```



```
        break;

        case "3":

            registrador.Registrar("Cerrando sesión");

            salir = true;

            break;

    }

}

}

static ConsoleColor CambiarColor(ConsoleColor colorActual)
{

    Console.WriteLine("\n=== Cambiar Color ===");

    Console.WriteLine("1. Rojo");

    Console.WriteLine("2. Verde");

    Console.WriteLine("3. Azul");

    Console.WriteLine("4. Amarillo");

    Console.WriteLine("5. Cian");

    Console.WriteLine("6. Magenta");

    Console.WriteLine("7. Blanco");

    Console.Write("\nElige un color: ");

    string opcion = Console.ReadLine();

    switch (opcion)
```

```
{  
  
    case "1": return ConsoleColor.Red;  
  
    case "2": return ConsoleColor.Green;  
  
    case "3": return ConsoleColor.Blue;  
  
    case "4": return ConsoleColor.Yellow;  
  
    case "5": return ConsoleColor.Cyan;  
  
    case "6": return ConsoleColor.Magenta;  
  
    case "7": return ConsoleColor.White;  
  
    default: return colorActual;  
  
}  
  
}  
  
}
```

Capturas

```
=== Sistema de Logging ===
```

```
Usuario: Aleeexx
```

```
Contraseña: *****
```

```
✓ Login exitoso
```

```
>
```

```
=== MENÚ ===
```

```
1. Ver registro
```

```
2. Cambiar color
```

```
3. Cerrar sesión
```

```
Opción: █
```

Usuario: Aleexx

Contraseña: *****

✓ Login exitoso

=== MENÚ ===

1. Ver registro
2. Cambiar color
3. Cerrar sesión

Opción: 2

=== Cambiar Color ===

1. Rojo
2. Verde
3. Azul
4. Amarillo
5. Cian
6. Magenta
7. Blanco

Elige un color: 1

[02:02:45] [Aleexx] >>> Color cambiado a Red

=== MENÚ ===

1. Ver registro
2. Cambiar color
3. Cerrar sesión

Opción: █

```
=== Sistema de Logging ===  
❖ Usuario: Juan  
Contraseña: *****  
  
✓ Login exitoso  
  
=== MENÚ ===  
1. Ver registro  
2. Cambiar color  
3. Cerrar sesión  
  
Opción: █
```

```
Program.cs logs.txt X  
logs.txt  
1 [2025-11-12 13:01:01] Sistema funcionando correctamente  
2 [2025-11-12 13:06:44] Color cambiado a Red  
3 [2025-11-12 13:06:48] Cerrando sesión  
4 [2025-11-12 13:08:10] Color cambiado a Red  
5 [2025-11-12 13:08:13] Cerrando sesión  
6 [2025-11-12 17:32:41] Color cambiado a Yellow  
7 [2025-11-12 17:32:53] Visualizando registro del sistema  
8 [2025-11-12 17:33:01] Cerrando sesión  
9 [2025-11-18 02:02:45] Color cambiado a Red  
10 [2025-11-18 02:03:07] Cerrando sesión  
11
```

Conclusion

El Decorator sirve para darle formato a los logs sin tocar el código base. Cada capa mete algo: color, fecha, usuario y símbolos. Se van apilando uno encima del otro, como cuando a un plato le vas agregando cosas sin cambiar el plato original.

El Adapter toma esos logs ya formateados y los guarda en un archivo. Actúa como un puente entre el sistema que muestra mensajes y el sistema que escribe en disco.

La idea es que cada patrón hace lo suyo. El Decorator se encarga de cómo se ven los logs; el Adapter se encarga de dónde terminan guardados. Si un día cambias el formato, solo mueves decoradores. Si quieres otra forma de guardar, cambias el adapter. Y ya, nada más.